

Kinds in Haskell and Their Analogues in F# and JavaScript

Jaanis Fehling — Università di Pisa

Abstract

Contents

1	Glossary of the Haskell Type System	2
2	Kinds in Haskell	3
2.1	Higher-kinded Types	4
2.2	Other Kinds	4
3	Kinds in F#	5
4	Kinds in JavaScript	6
5	Conclusion	7
A	Supplementary Material	7

1 Glossary of the Haskell Type System

Before investigating kinds, we briefly explain the most important definitions in Haskell's type system and make example(s).

Type

A type classifies values and expressions; it describes what kind of data something is and which operations are valid on it.

```
Int, Bool, Char
[Int]
int -> Bool
Maybe Int
```

Type Signature

A type signature is an explicit annotation of an expression's type.

```
not :: Bool -> Bool
```

Type Variable

A type variable is a placeholder for an unknown type, enabling polymorphism. Usually written `a`, `b`, `t`, etc.

```
id :: a -> a -- Here the type signature of "id" contains type variables
```

(Parametric) Polymorphism

Parametric polymorphism means code works uniformly for any type, without inspecting that type.

```
length :: [a] -> Int
```

Type Constructor

A type constructor is something that builds new types from type parameters (a "type-level function").

```
[] :: Type -> Type
Maybe :: Type -> Type
Either :: Type -> Type -> Type
```

(Data) Constructor

A data constructor builds values, not types.

```
Just :: a -> Maybe a
Nothing :: Maybe a
```

Type Class

A type class defines a set of operations (methods) and laws that a type can implement. It's like an interface/contract at the type level.

```
class Eq a where
    (==) :: a -> a -> Bool
```

Type Constraint

A type constraint restricts a type variable to types that are instances of some type class(es). Written before `=>` in a type signature.

```
(==) :: Eq a => a -> a -> Bool
foo :: (Eq a, Show a) => a -> String
```

Constrained Polymorphism

When a function is polymorphic but only for types satisfying constraints (type classes), it's constrained (often called ad-hoc polymorphism).

```
show :: Show a => a -> String
```

2 Kinds in Haskell

Haskell offers besides a classical typesystem a "typesystem of a typesystem", also called *kinds* (i.e. kind of a type or type of a type). In GHCi, the interactive haskell interpreter, we can use the `:t` command to check the type of a previously declared variable:

```
ghci> x = 42 :: Int
ghci> :t x
x :: Int
```

Now, the type `Int` has the following kind, which we can check using the `:k` command:

```
ghci> :k Int
Int :: *
```

In contrast, we can check the kind of a type constructor such as `Maybe`:

```
ghci> :k Maybe
Maybe :: * -> *
```

`Maybe` is a type constructor, meaning it takes an additional type as an argument, which will then produce the kind:

```
ghci> :k Maybe Int
Maybe Int :: *
```

More complex kinds are also possible:

```
ghci> :k Either
Either :: * -> * -> *
ghci> :k (->)
(->) :: * -> * -> *
```

Under the hood GHC, the Haskell Compiler, treats kinds as types, meaning internally mostly the same procedures are used to check the correctness of types and kinds.

2.1 Higher-kinded Types

A higher-kinded type is a type constructor taking a type constructor as an argument:

```
ghci> :k Functor
Functor :: (* -> *) -> Constraint
```

Here `Constraint` is a the kind of type constraints. We can test the `Functor` kind by supplying different types or type constructors:

```
ghci> :k Functor Maybe
Functor Maybe :: Constraint
ghci> :k Functor Int
```

```
<interactive>:1:9: error:
  - Expected kind '* -> *', but 'Int' has kind '*'
  - In the first argument of 'Functor', namely 'Int'
  In the type 'Functor Int'
```

Higher-kinded types allow the programmer to abstract over type constructors. For example, we can write a utility function that converts to elements implementing the `Show` typeclass to strings in any (data) structure implementing the `Functor` typeclass:

```
liftAndShow :: (Functor f, Show a) => f a -> f String
liftAndShow = fmap show
```

This works only with higher-kinded types because `Functor` needs a higher-kinded parameter:

```
class Functor (f :: * -> *) where
  fmap :: (a -> b) -> f a -> f b
```

Another example is the function `sequence` that evaluates every action in a list of monads and collects the results:

```
sequence :: Monad m => [m a] -> m [a]
```

Here, `m` is higher-kinded: `m :: * -> *`.

2.2 Other Kinds

DataKinds

With `DataKinds`, constructors can be used as types.

```
{-# LANGUAGE DataKinds #-}
import Data.Kind (Type)

data Nat = Z | S Nat
```

Now, new types or type constructors are created for the constructors, prefixed by `'`:

```
Nat :: *
'Z :: Nat
'S :: Nat -> Nat
```

PolyKinds

With PolyKinds, GHC can infer kind-polymorphic definitions.

```
{-# LANGUAGE PolyKinds #-}
data Proxy (a :: k) = Proxy
```

Here `k` is a kind variable, and the constructor is kind-polymorphic:

```
Proxy :: forall k. k -> *
```

This is similar to polymorphic types, but at the kind level.

3 Kinds in F#

Haskell's kind system is similar to the arity of type constructors in F#. In F#, generic type definitions are most similar to Haskell's type constructors and parametric polymorphism:

```
int // no parameters
'T option // one parameter
Result<'T, 'E> // two parameters
```

There is no distinct kind or type-of-types system, but different generic types take different number of type arguments.

Where F# lacks in comparison to Haskell is higher-kinded types. There is no polymorphism over type constructors. We cannot express, at the type level, that a generic parameter is itself a type constructor of a particular arity (e.g. something that takes a type argument), in Haskell this is expressed and enforced by the kind system. We cannot, for example, write a generic implementation of a `map` function for any type unary type constructor. We can write the following in F#:

```
let incList xs = List.map ((+) 1) xs
let incOption x = Option.map ((+) 1) x
```

But not:

```
let incAny (x : F<int>) = map ((+) 1) x
```

Generics in F# only work for types, not type constructors. Furthermore, there is no equivalent to Haskell's `DataKinds` and `PolyKinds`.

A Workaround that is usable in F# is representing the higher-kinded type using a generic type with two type variables, with the first type variable being a "witness" type, indicating what type constructor it is, for example `Option` or `List`, and the second type variable being the element type, for example `int`. The result is a type like `App<'F, 'T>`, which during runtime is just a boxed object. Because this is a runtime solution to a static typing problem, the construct does not enforce that the object contains the correct type specified in the witness type variable. An example code snippet:

```
// HKT encoding: "F applied to T"
type App<'F, 'T> = private App of obj

// Example tags
type OptionTag = class end
type ListTag = class end

module HKT =
```

```

let inline inj (x: 'Real) : App<'F, 'T> = App (box x)
let inline prj (App o) : 'Real = unbox o

type Functor<'F> =
{ Map : ('a -> 'b) -> App<'F, 'a> -> App<'F, 'b> }

// Instances
let optionFunctor =
{ Map =
  fun f fa ->
    let x : option<_> = HKT.prj fa
    HKT.inj (Option.map f x) }

let listFunctor =
{ Map =
  fun f fa ->
    let x : list<_> = HKT.prj fa
    HKT.inj (List.map f x) }

```

Here, the `inj` and `prj` functions allow to pack or unpack the actual objects into the encoded representation (`App<'F, 'T>`).

Another approach is to wrap values in an object that exposes the operations we need, such as `map` in our example. We create a common interface, for example an interface resembling functors:

```

type IFunctor<'T> =
  abstract Map<'U> : ('T -> 'U) -> IFunctor<'U>

type OptionF<'T>(x: option<'T>) =
  member _.Value = x
  interface IFunctor<'T> with
    member _.Map f = OptionF(Option.map f x) :> IFunctor<_>

type ListF<'T>(x: list<'T>) =
  member _.Value = x
  interface IFunctor<'T> with
    member _.Map f = ListF(List.map f x) :> IFunctor<_>

```

Usage of the wrapper objects:

```

let a : IFunctor<int> = OptionF(Some 1) :> IFunctor<_>
let b : IFunctor<string> = a.Map string

```

This approach is what would be used in standard object-oriented programming. One downside is that you have to wrap every type constructor with the interface, stacking abstractions such as `Functor`, `Applicative` and `Monad` like in Haskell is not possible without writing many interfaces. Additionally, you program against a interface (`IFunctor<'U>`), so you cannot statically know if it is `OptionF<'U>` or `ListF<'U>`.

4 Kinds in JavaScript

JavaScript has also no kind system, it does not even have a static type system. To achieve kind-like reasoning, we can employ the same methods previously described for F# in

TypeScript. For the first variant using a generic type with two type variables, one being a "witness/tag" type and the other type variable being the element type, we can model it in TypeScript:

```
interface URIToKind<A> {
  Array: Array<A>;
  Option: Option<A>;
}
type URIS = keyof URIToKind<any>;

type Kind<URI extends URIS, A> = URIToKind<A>[URI];

interface Functor<URI extends URIS> {
  readonly URI: URI;
  map: <A, B>(fa: Kind<URI, A>, f: (a: A) => B) => Kind<URI, B>;
}

const arrayFunctor: Functor<"Array"> = {
  URI: "Array",
  map: (fa, f) => fa.map(f),
};
```

5 Conclusion

A Supplementary Material